Computer Networks

3. Transport Layer 5

Transport Layer: Overview

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Principles of reliable data transfer
- Connection-oriented transport: TCP
- Principles of congestion control
- TCP congestion control
- Evolution of transport-layer functionality

TCP 혼잡 제어: AIMD

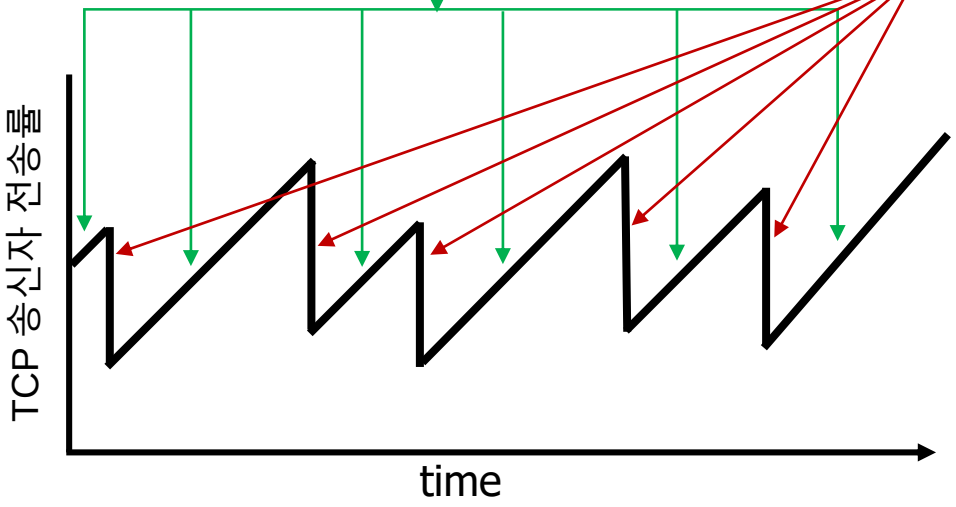
- 접근법: 송신자는 패킷 손실 (혼잡)이 발생할 때까지 전송 속도를 높이고, 손실 발생 시 전송률을 낮춤

가법적 증가 (*Additive Increase*)

손실이 감지될 때까지, RTT마다 최대 세그먼트 크기 '하나' 씩 전송률을 증가

승법적 감소 (*Multiplicative Decrease*)

손실 감지되면 전송률을 '절반'으로 줄임



AIMD의 톱니 동작:

- 대역폭 탐색

TCP 혼잡 제어: AIMD

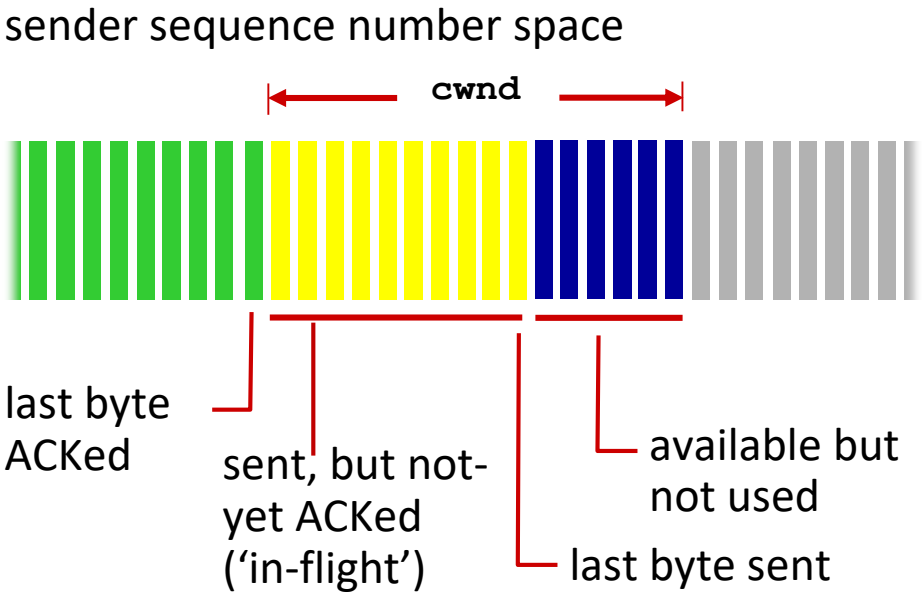
승법적 감소 (Multiplicative decrease)에서 전송률:

- 타임아웃으로 패킷 손실이 감지되면 1MSS (최대 세그먼트 크기)으로 줄임 (TCP *Tahoe*, 1988)
- 3개의 중복 ACK를 받으면, 절반으로 줄임 (TCP *Reno*, 1990)

왜 AIMD 인가?

- AIMD는 분산적이며 비동기적인 알고리즘으로,
  - 네트워크 전반에서 혼잡 상황에 따른 전송 윈도우 크기를 효과적으로 조절할 수 있음
  - 공평성과 안정성을 갖추고 있어, 다양한 네트워크 환경 변화에도 유연하게 대응할 수 있음

TCP 혼잡 제어: 상세



$$\text{LastByteSent} - \text{LastByteAcked} \leq \text{cwnd}$$

윈도우 사이즈: $N = min\ (rwnd,\ cwnd)$

$$\text{TCP 전송률} \approx \frac{\text{cwnd}}{\text{RTT}}\ \text{bytes/sec}$$

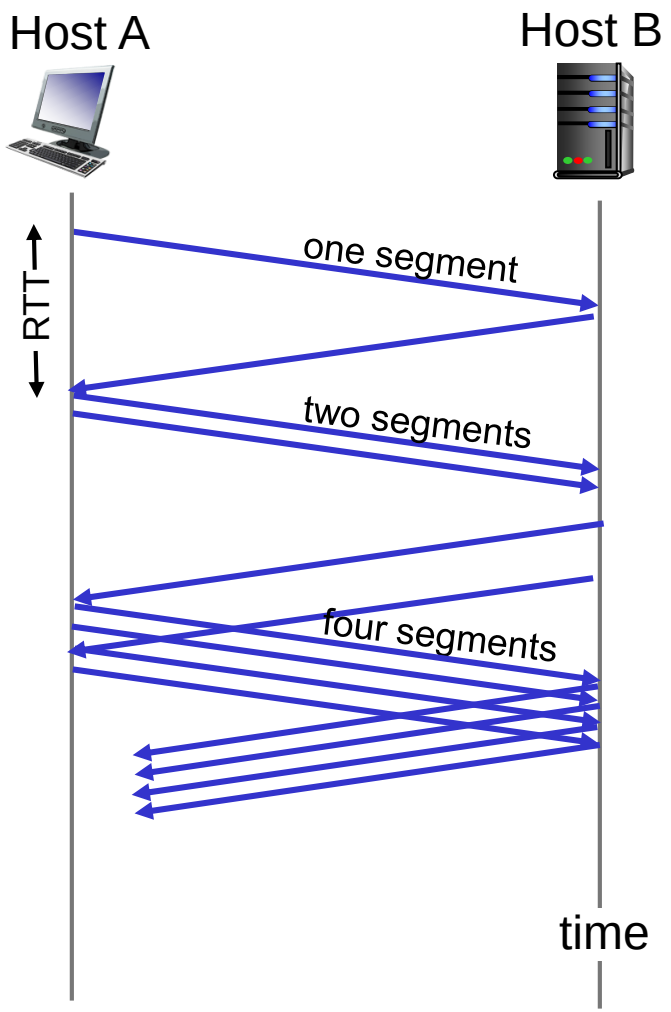
- 혼잡 윈도우 (congestion window, cwnd)는 TCP 송신자는 전송을 제한
- cwnd는 탐지된 네트워크 혼잡에 따라 동적으로 조정됨 (TCP의 혼잡 제어 구현)

TCP의 대략적인 전송 동작:

- cwnd 만큼 바이트를 보내고, 확인응답을 위해 RTT 만큼 기다린 다음, cwnd 값을 조절하면서 더 많은 바이트를 전송

TCP 슬로 스타트 (slow start)

- TCP 연결을 시작했거나 타임 아웃이 발생했을 때 '슬로 스타트' 시작
  - **cwnd** = 1 MSS로 초기화
- 이후 매 RTT마다 **cwnd** 값은 두배 씩 증가
  - ACK가 수신될 때 마다, cwnd = cwnd + MSS
  - **[예]** 현재 cwnd = 4이면, 4개의 세그먼트를 전송하고 RTT 후 4개의 ACK를 성공적으로 수신한다면, 다음 cwnd: 현재 cwnd (4) + ACK (4) = 8
- 요약: 초기 속도는 느리지만, 기하급수적 (두배 씩)으로 빠르게 증가!



TCP 슬로 스타트에서 혼잡 회피로 전환

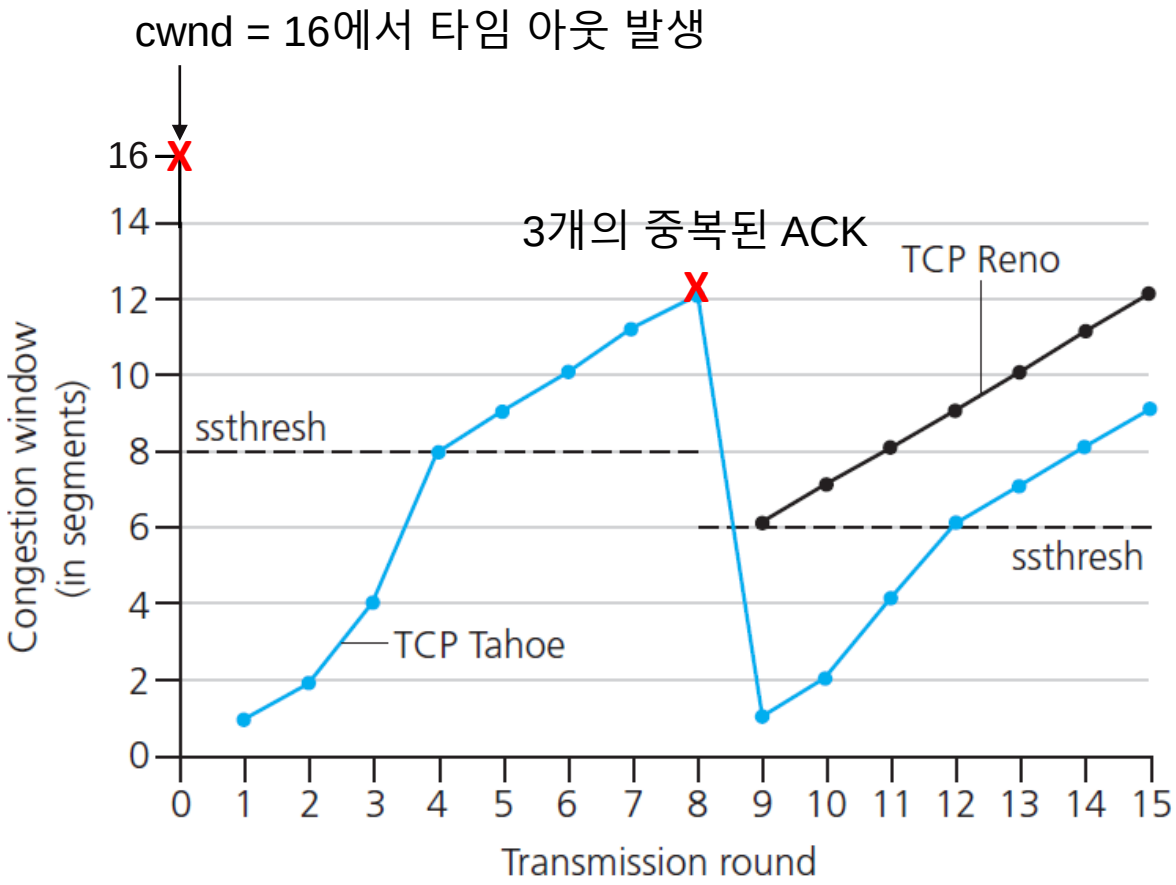
Q: 언제까지 기하급수적으로 증가할 수 있을까?

A: **cwnd**가 슬로 스타트의 임계 값 (slow start threshold, ssthresh)에 도달할 때까지

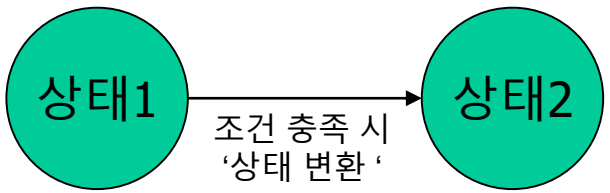
- ssthresh: 혼잡이 마지막으로 탐지된 시점에서의 **cwnd** 값의 절반

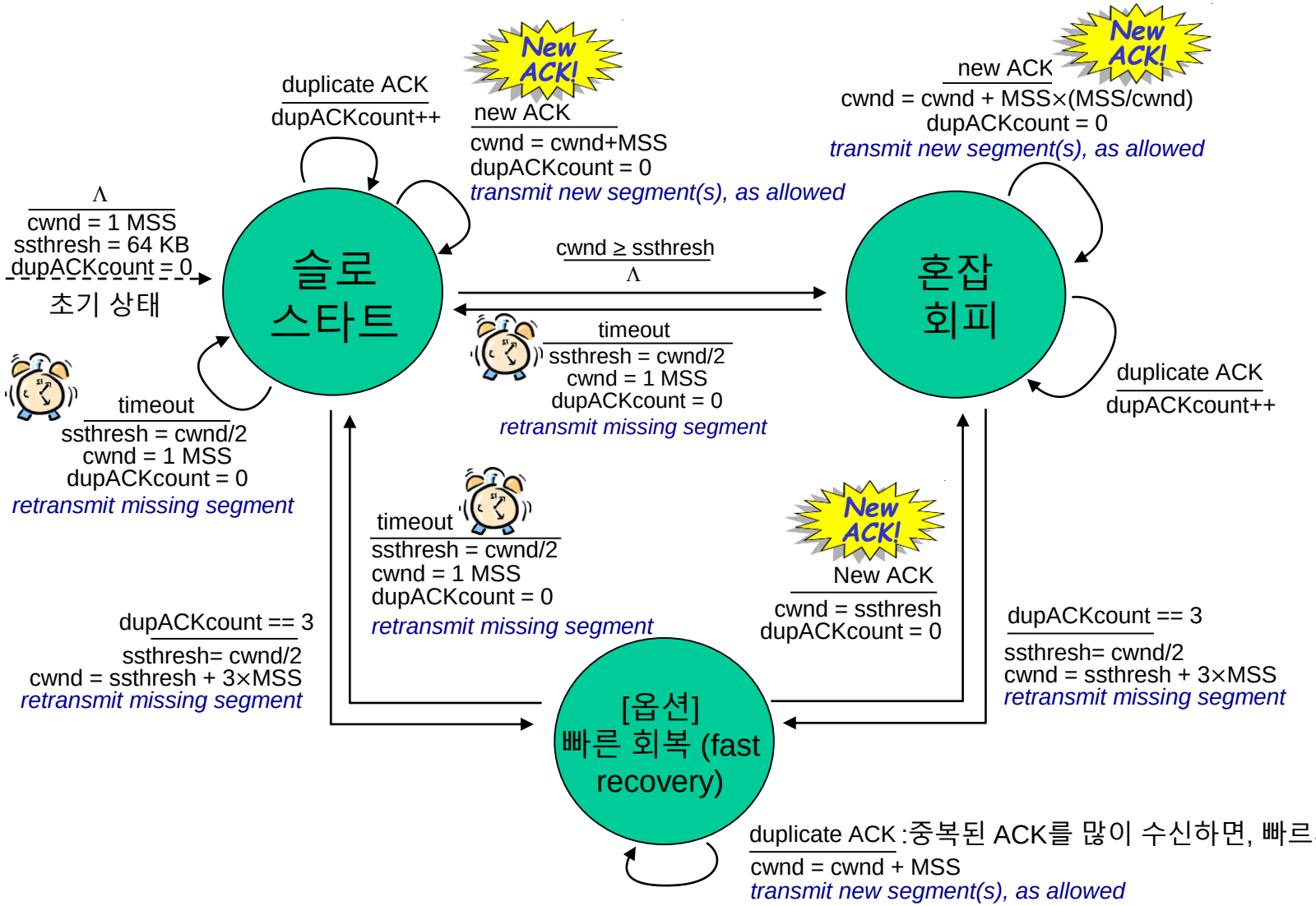
혼잡 회피 (congestion avoidance)

- **cwnd** 값이 **ssthresh**에 도달하면, 슬로 스타트가 종료되고 혼잡 회피 시작
- '전체' **cwnd**에 대한 확인응답이 되면, **cwnd** 값을 하나씩 증가 (선형적으로 증가)



[TMI] TCP Reno 혼잡 제어 요약

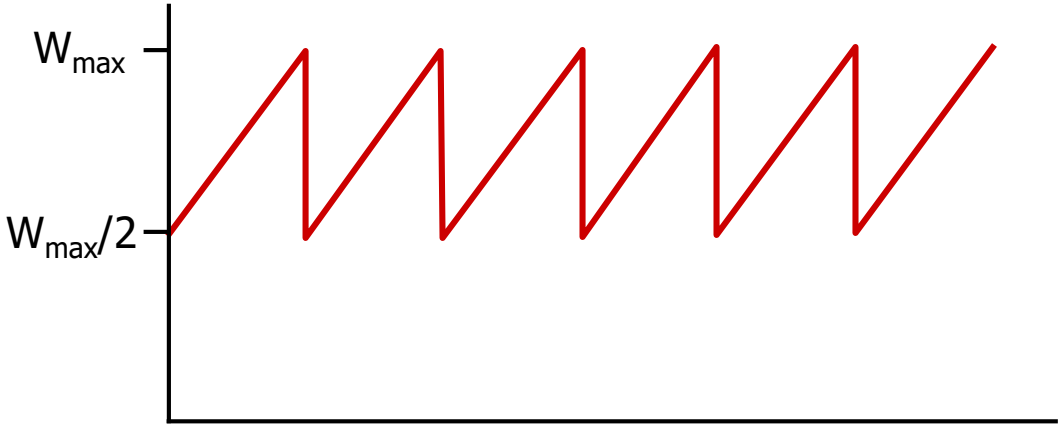




Λ: 발생한 이벤트가 없다는 의미 또는 어떤 이벤트가 발생해도 아무 액션을 하지 않는다는 의미

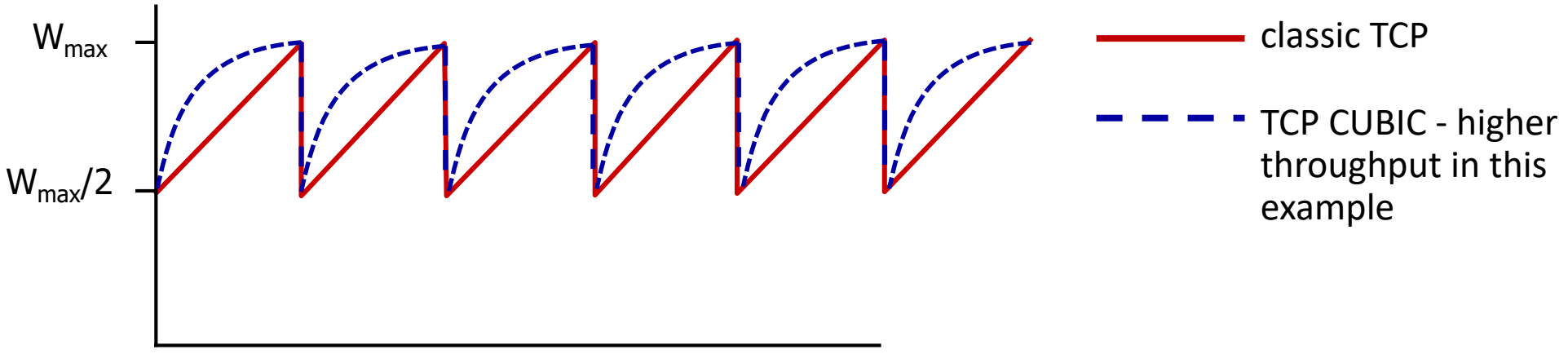
TCP Reno의 거시적 처리율

- TCP의 거시적인 처리율은 혼잡 윈도우 크기와 RTT의 관계로 나타낼 수 있음
- W_{max}: 손실이 발생하는 윈도우의 크기 (바이트 단위)
  - 가정: 항상 보낼 데이터가 있으며, 슬로 스타트는 무시하고, W_{max}는 크게 변하지 않음
  - 평균 윈도우 크기는 $\frac{3}{4} W_{max}$ → 평균 TCP 처리율 = $\frac{3}{4} \frac{W_{max}}{RTT}$ bytes/sec



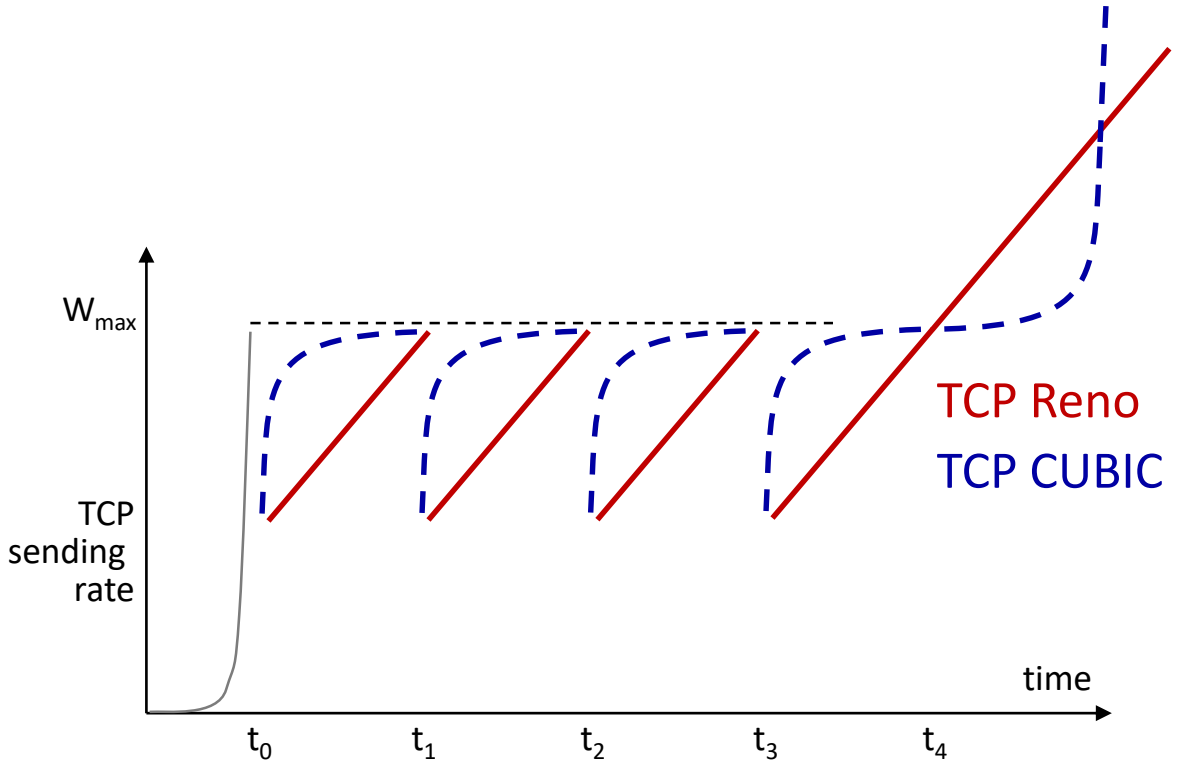
TCP CUBIC

- 사용 가능한 대역폭을 '탐색' 하는 데 AIMD보다 더 좋은 방법은?
- Insight/intuition:
  - W_{max}: 손실이 마지막으로 감지되었을 때 혼잡 윈도우 크기
  - 패킷 손실이 발생한 병목 링크의 상태는 아마도(?) 크게 변경되지 않았음
  - 손실 발생시 혼잡 윈도우 크기를 절반으로 줄인 후, 처음에는 W_{max}에 더 빠르게 접근하지만 나중에는 W_{max}에 더 느리게 접근



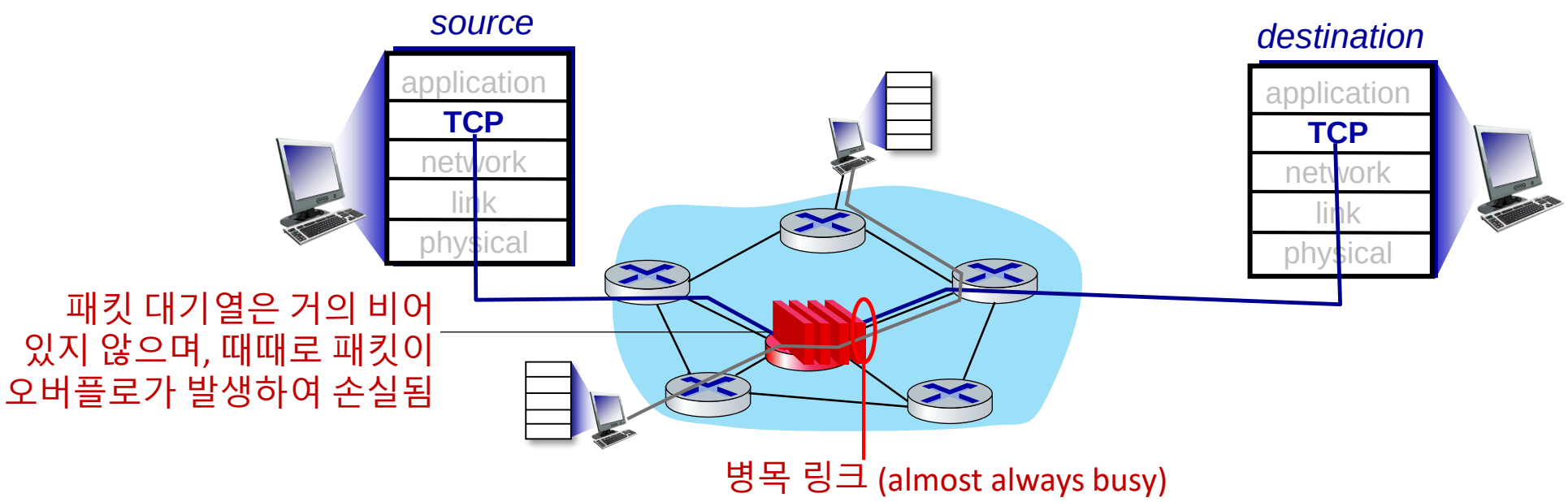
TCP CUBIC

- *K*: TCP 윈도우 크기가 W_{max}에 도달하는 미래 시점
  - *K* 자체는 조정 가능
- 현재 시각 (*t*)와 *K* 시각 사이의 거리의 세제곱 함수로 혼잡 윈도우 크기 (W)를 증가
  - *K*에서 멀어지면 더 빠르게 증가
  - *K*에 가까울수록 더 천천히 증가 (주의)
- 리눅스의 기본 TCP 방식이며, 웹서버에서도 가장 많이 사용되는 TCP 방식임



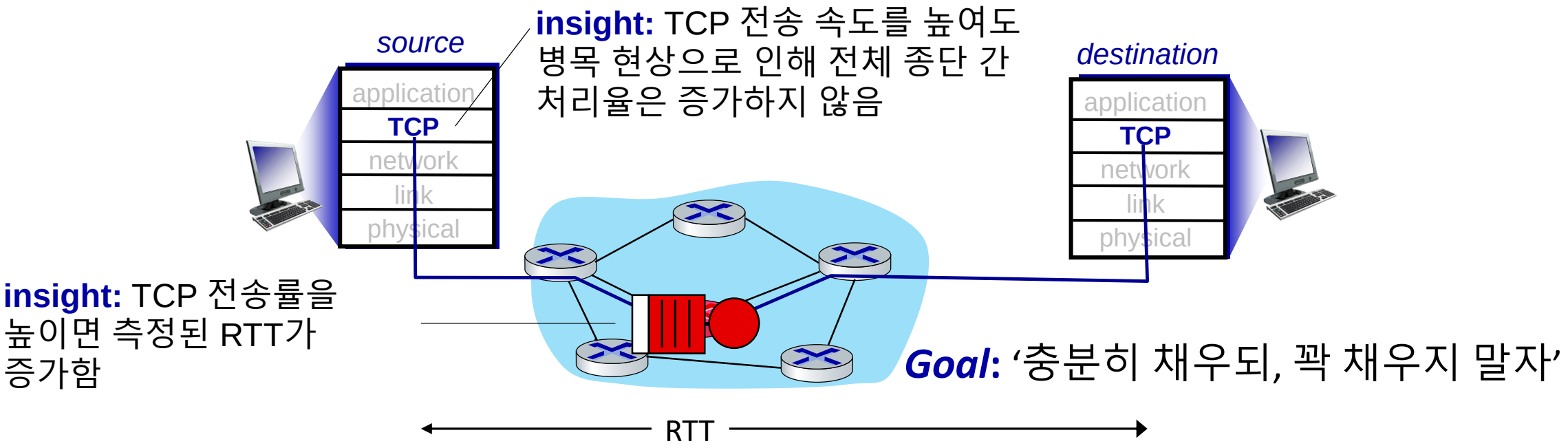
TCP와 혼잡한 '병목링크'

- 손실 기반 (loss-based) 혼잡 제어: 패킷 손실을 혼잡으로 판단
- TCP (classic, CUBIC)는 일부 라우터의 출력 (병목 링크)에서 패킷 손실이 발생할 때까지 TCP의 전송 속도를 높임



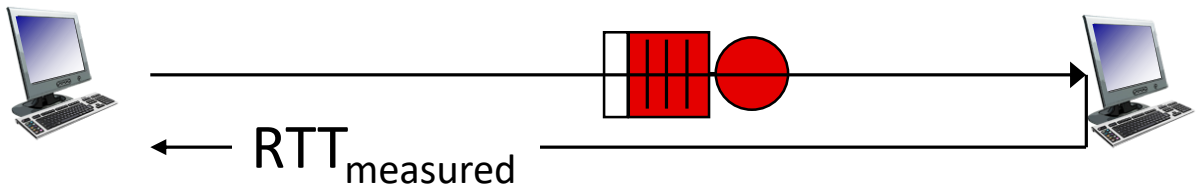
TCP와 혼잡한 '병목링크'

- 손실 기반 (loss-based) 혼잡 제어: 패킷 손실을 혼잡으로 판단
- TCP (classic, CUBIC)는 일부 라우터의 출력 (병목 링크)에서 패킷 손실이 발생할 때까지 TCP의 전송 속도를 높임
- 혼잡의 이해: 혼잡이 발생한 병목 링크에 초점을 맞춤



지연 기반 (Delay-based) 혼잡 제어

송신자와 수신자 간의 파이프를 '충분히 채우되, 꽉 채우지 말자': 병목 링크를 전송 중인 상태로 계속 유지 (처리율의 최대화), 하지만 긴 지연 시간과 버퍼링은 피해야 됨 (지연을 낮게 유지하면서)



$$\text{처리율 측정} = \frac{\text{마지막 RTT 간격에 전송된 바이트 수}}{RTT_{measured}}$$

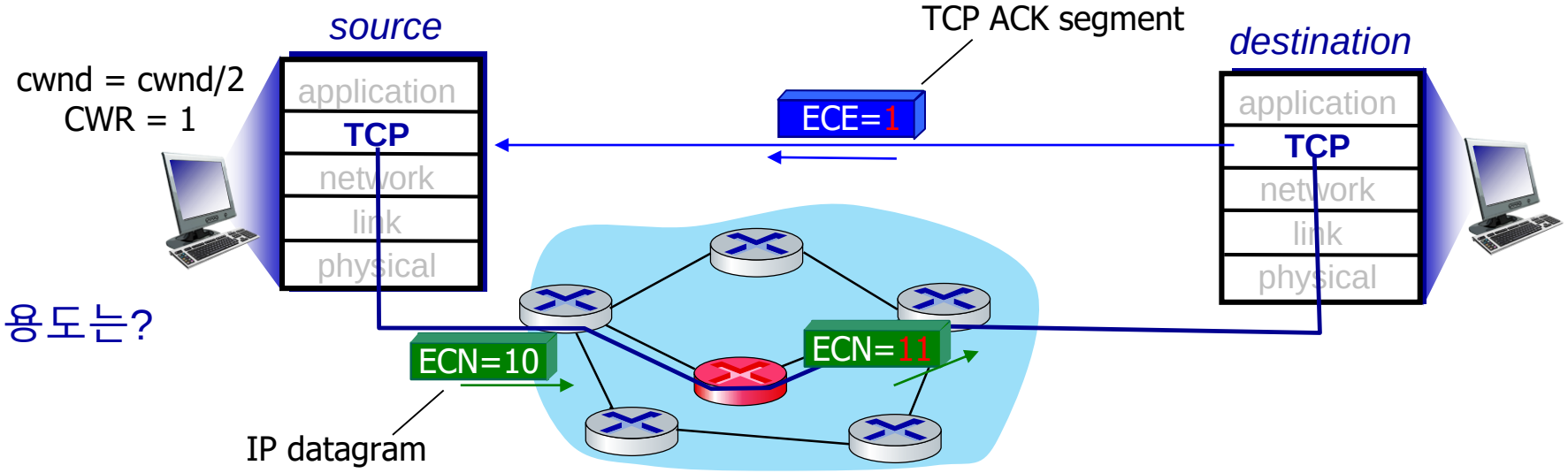
지연 기반 혼잡 제어 (TCP vegas, 1995): 손실을 유발하지 않고, 혼잡을 제어

- RTT_{min} : 송신자가 측정한 RTT의 최솟값 (혼잡하지 않고, 패킷의 최소 대기 시간)
- 혼잡하지 않는 처리율: cwnd/RTT_{min}
- 만약 측정된 처리율이 혼잡하지 않은 처리율에 '매우 근접'하면, 혼잡이 발생하지 않았다고 판단하고 cwnd를 선형적으로 증가
- 그렇지 않고, 혼잡하지 않은 처리율보다 '훨씬 아래'로 측정되면, 혼잡이 발생했다고 판단하고 cwnd를 선형적으로 감소

명시적 혼잡 알림 (Explicit Congestion Notification, ECN)

최근 배포된 TCP 일부는 네트워크 지원 혼잡 제어를 구현함:

- 혼잡이 발생한 라우터는 IP 헤더의 2비트 (type of service 필드의 ECN 비트)를 통해 혼잡 발생을 표시
  - 라우터의 혼잡 기준은 라우터 공급업체 및 네트워크 운영자가 결정
- 혼잡 표시가 수신자에게 전달되며, 수신자는 ACK 세그먼트 헤더의 ECE 비트를 1로 설정하여 송신자에게 혼잡이 발생 했음을 알림
- involves both IP (IP header ECN bit marking) and TCP (TCP header CWR, ECE bit marking)

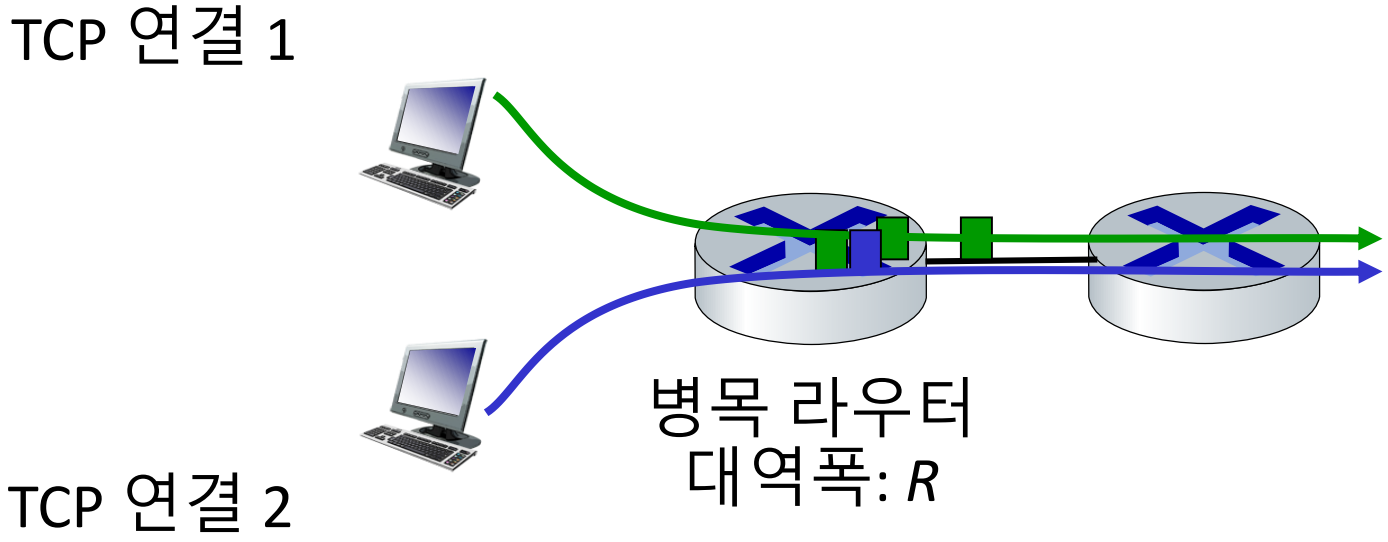


CWR bit의 용도는?

ECE: Explicit Congestion notification Echo, CWR: Congestion Window Reduced

TCP 공평성

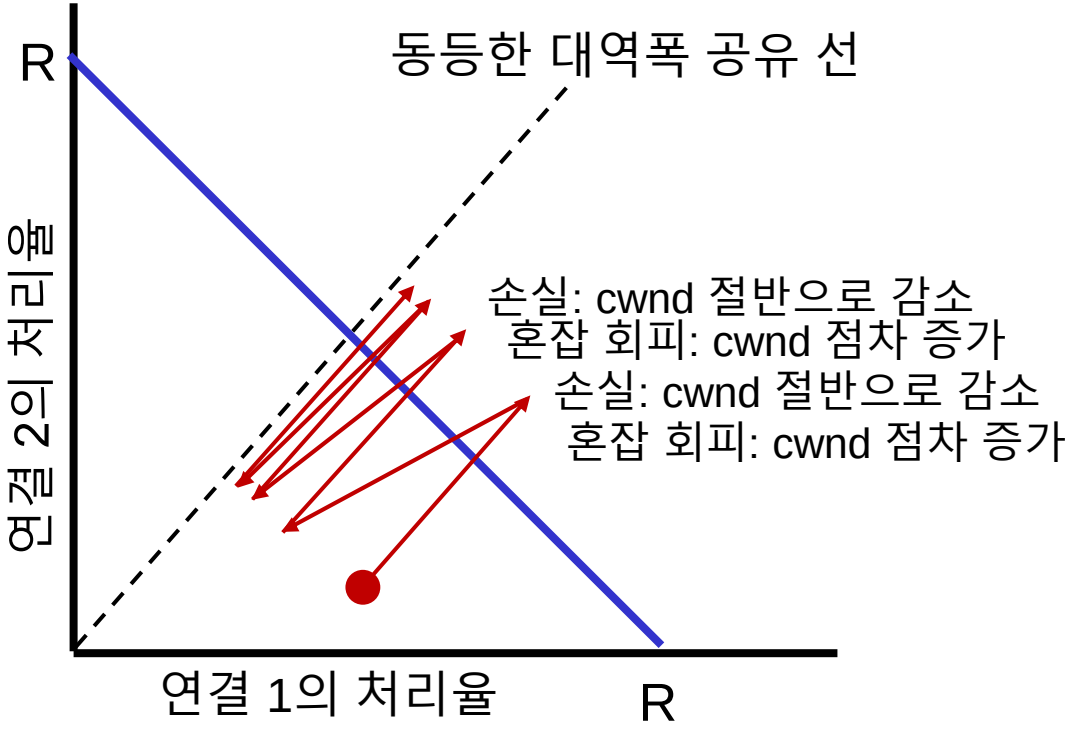
공평성 (Fairness) 목표: *K*개의 TCP 연결이 대역폭이 *R*인 동일한 병목 링크를 공유하는 경우, 각 연결의 평균 전송률은 *R/K*이 되어야 함



Q: 과연 TCP는 공평한가?

[예] 두개의 경쟁 TCP 연결:

- 가법적 증가 (additive increase)는 기울기가 1 (45°)이고 전체적으로 처리율을 증가시킴
- 승법적 감소 (multiplicative decrease)는 처리율을 비례적(절반)으로 감소시킴



Is TCP fair?

A: 이상적인 시나리오에서, TCP는 공평함:

- RTT 및 MSS 동일
- 연결 숫자는 고정이며, 언제나 혼잡 회피 방식으로만 동작

모든 네트워크 애플리케이션이 '공평' 할까?

공평성과 UDP

- 많은 멀티미디어 애플리케이션에서 TCP를 사용하지 않음
  - 혼잡 제어로 전송률이 조절되는 것을 원하지 않음
- 대신 UDP를 사용:
  - 일정한 속도로 오디오/비디오 전송이 가능하며, 패킷 손실을 허용
- TCP 관점에서 UDP는 공평하지 않음

공평성과 병렬 TCP 연결

- 애플리케이션이 두 호스트 간에 여러 병렬 TCP 연결을 설정할 수 있음
- [예] 9개의 클라이언트/서버 응용이 TCP 연결을 하나씩 설정하여 전송률 R의 링크를 공유하고 있음
  - 새로운 응용이 TCP 연결을 1개를 설정하고 R/10 전송률을 가짐
  - 새로운 응용이 TCP 연결을 병렬적으로 9개를 설정하여 R/2 전송률을 가짐

Transport Layer: Overview

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Principles of reliable data transfer
- Connection-oriented transport: TCP
- Principles of congestion control
- TCP congestion control
- Evolution of transport-layer functionality

[TMI] 트랜스포트 계층의 기능의 발전

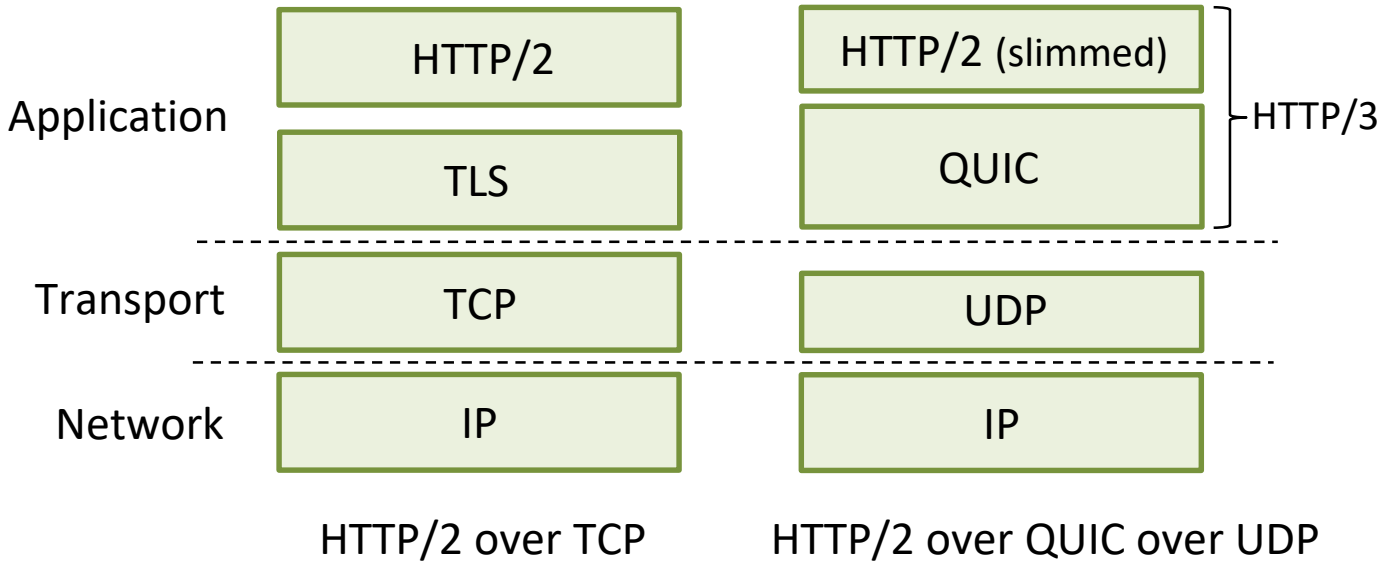
- TCP, UDP: 40년간의 주요 전송 프로토콜
- 특정 시나리오를 위해 개발된 다양한 버전의 TCP:

Scenario	Challenges
Long, fat pipes (large data transfers)	Many packets ‘in flight’; loss shuts down pipeline
Wireless networks	Loss due to noisy wireless links, mobility; TCP treat this as congestion loss
Long-delay links	Extremely long RTTs
Data center networks	Latency sensitive
Background traffic flows	Low priority, ‘background’ TCP flows

- 트랜스포트 계층의 기능을 UDP 위에 있는 애플리케이션 계층으로 이동
  - HTTP/3: QUIC

QUIC: Quick UDP Internet Connections (빠른 UDP 인터넷 연결)

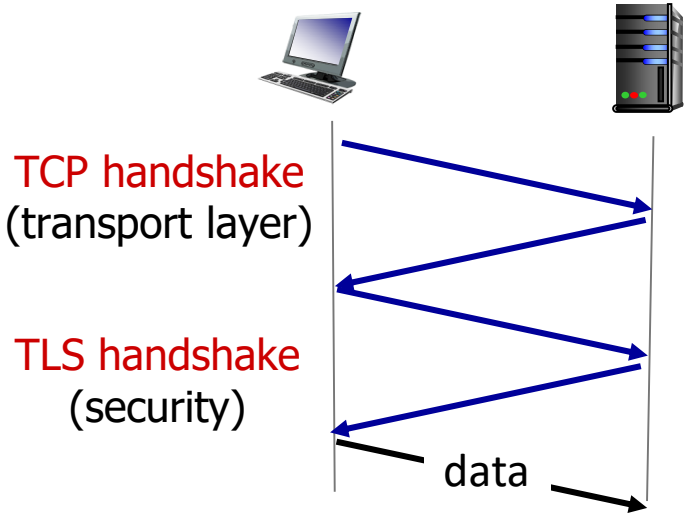
- UDP를 하위 트랜스포트 계층 프로토콜로 사용하는 애플리케이션 계층 프로토콜
  - 보안 HTTP를 위한 트랜스포트 계층 서비스의 성능 향상을 위해 설계됨
  - deployed on many Google servers, apps (Chrome, mobile YouTube app)

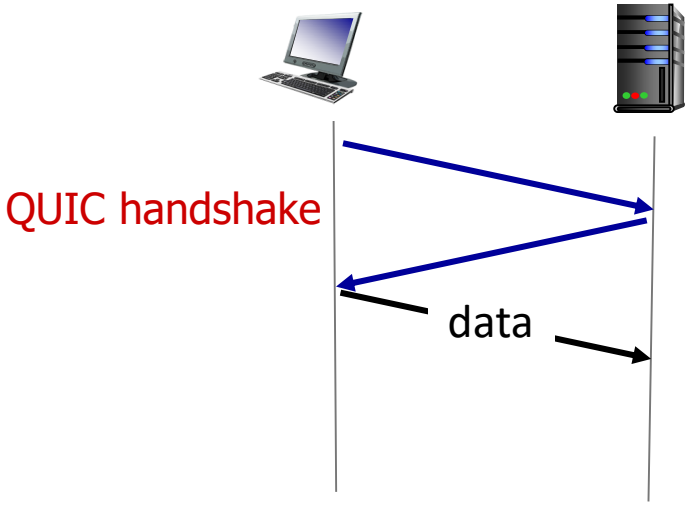


QUIC: Quick UDP Internet Connections (빠른 UDP 인터넷 연결)

- 오류 및 혼잡 제어 (error and congestion control): ‘TCP의 손실 감지 및 혼잡 제어에 익숙한 독자는 잘 알려진 TCP 알고리즘과 유사한 알고리즘을 여기서 찾을 수 있다.’ (QUIC 명세서 초안)
- 연결 설정 (connection establishment): 하나의 RTT 동안, 신뢰성, 혼잡 제어, 인증, 암호화, 상태 설정을 수행
  - 출발지와 목적지 ‘연결 ID’ 기반으로 연결을 구분
- 단일 QUIC 연결을 통해 여러 애플리케이션 레벨의 스트림들 (streams)을 다중화
  - 각 객체는 ‘스트림 ID’로 구분되며, 같은 세그먼트에 여러 스트림 데이터가 포함될 수 있음
  - 각 스트림 별로 신뢰적인 데이터 전송, 보안을 제공
  - common congestion control (new Reno 기반)

QUIC: Connection establishment





TCP (reliability, congestion control state) + TLS (authentication, crypto state)

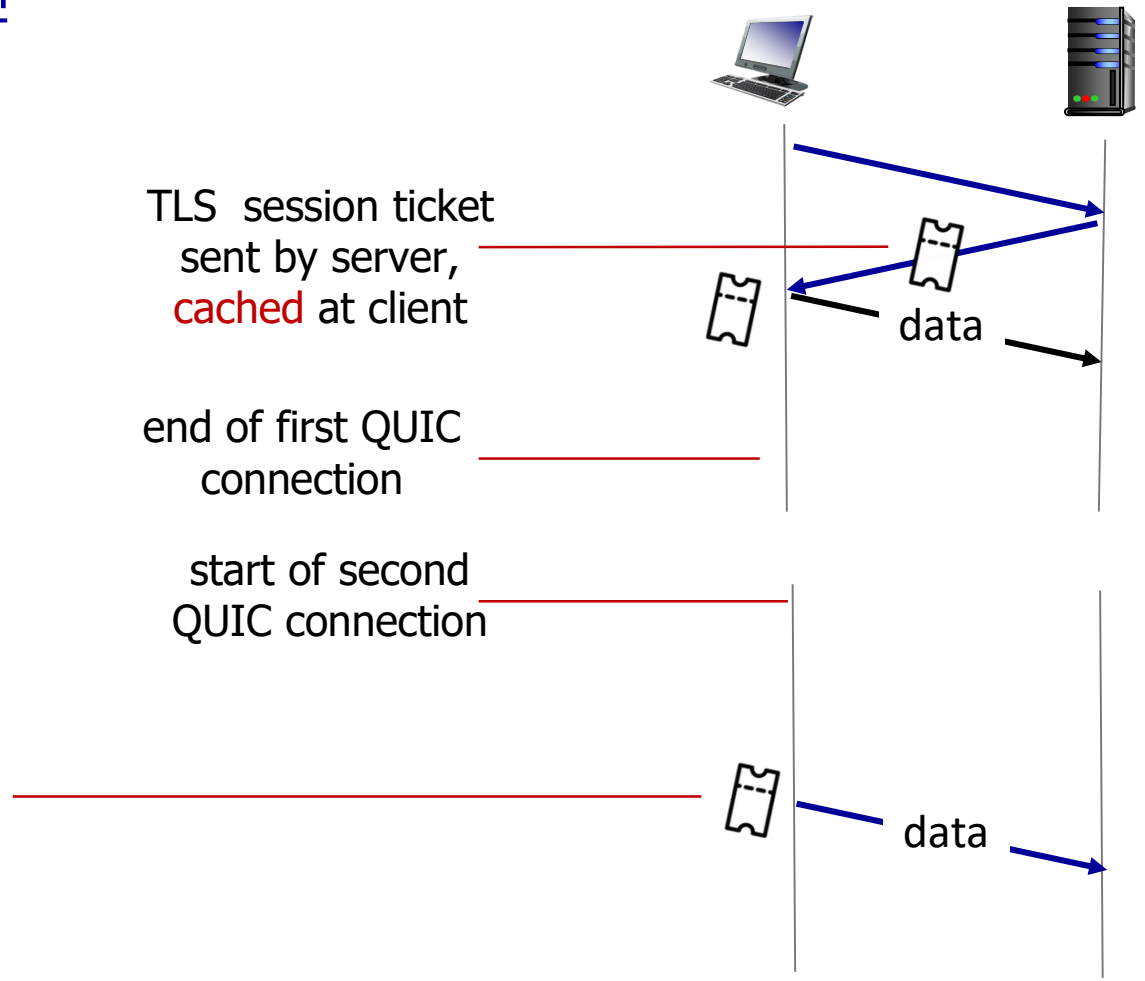
- 2 번의 연속적인 handshakes

QUIC: reliability, congestion control, authentication, crypto state

- 1 handshake
  - 출발지와 목적지의 연결 ID 활용

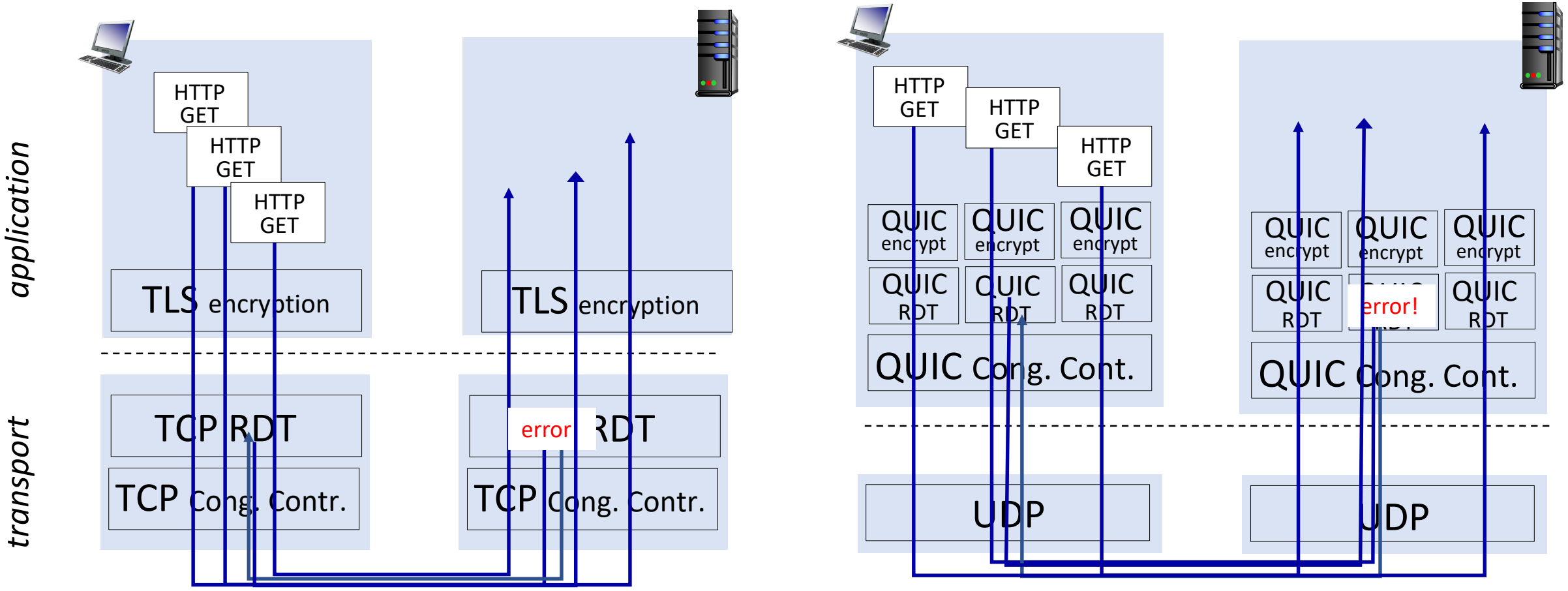
QUIC: 0-RTT Connection establishment

- 클라이언트는 이전 연결에서 받은 세션 티켓을 캐싱함
- 다음 연결에서는 0-RTT 데이터를 즉시 전송할 수 있음
  - 0-RTT는 지연을 줄이지만, replay 공격 가능성은 남아 있음



Client uses last session ticket for encryption, authentication for new connection

QUIC: streams - parallelism, no HOL blocking



(a) HTTP 1.1

(b) HTTP/2 with QUIC: no HOL blocking

Transport layer: summary

understand principles of transport layer services

- logical communication between processes
- multiplexing and socket
  - multiplexing and de-multiplexing
  - socket and socket program
- UDP: user datagram protocol
  - connectionless service
  - fast but unreliable transfer
- reliable data transfer principles
  - handling of bit error and packet loss
  - stop-and-wait, go-back-N, selective-repeat

Transport layer: summary

- TCP: transmission control protocol
  - connection-oriented service
  - reliable data transfer though error control and flow control
- congestion control
  - congestion: ‘load is higher than network capacity!’
  - additive increase and multiplicative decrease
- TCP congestion control algorithm
  - dynamics of congestion window
  - TCP tahoe, TCP reno

Transport layer: summary

up next:

- leaving the network ‘edge’ (application, transport layers)
- into the network ‘core’
- two network-layer chapters:
  - data plane
  - control plane

Q & A